30 Cybersecurity Projects Using Machine Learning

Complete Guide with Datasets & Evaluation Rubric

Document Purpose: This comprehensive guide provides 30 cybersecurity machine learning project ideas, required team deliverables, and a detailed evaluation rubric.

Table of Contents

Part 1: 30 Cybersecurity ML Project Ideas with Dataset Links

Part 2: Required Outputs from Each Team

Part 3: Evaluation Rubric (Total: 20 Points)

Part 4: Scoring Summary & Grade Boundaries

Part 5: Submission Checklist & Timeline

PART 1: 30 Cybersecurity ML Projects with Datasets

Section A: Network Security & Intrusion Detection

1. Network Intrusion Detection System (NIDS)

- **Description:** Build a system to detect network intrusions and classify attack types using ML algorithms
- **Datasets:**
  - NSL-KDD Dataset (Kaggle)
  - CIC-IDS2017 (Kaggle)
  - UNSW-NB15 (UNSW Canberra)

2. DDoS Attack Detection & Prevention

- **Description:** Detect Distributed Denial of Service attacks in real-time network traffic
- **Datasets:**
  - CIC-DDoS2019 (UNB)
  - DDoS Attack Dataset (Kaggle)

3. Botnet Detection System

- **Description:** Identify compromised devices forming botnets through traffic pattern analysis

- **Datasets:**
    - CTU-13 Botnet Dataset
    - Botnet Traffic Dataset (Kaggle)

4. Network Anomaly Detection

- **Description:** Detect unusual network behavior patterns indicating potential threats
- **Datasets:**
    - KDD Cup 99 (UCI)
    - NF-UNSW-NB15-v2 (Kaggle)

5. Zero-Day Attack Detection

- **Description:** Identify previously unknown attacks using unsupervised/deep learning techniques
- **Datasets:**
    - Zero-Day Attack Detection (Kaggle)
    - UNSW-NB15 (UNSW Canberra)

Section B: Malware Analysis & Detection

6. Malware Classification System

- **Description:** Classify malware families using static and dynamic analysis features
- **Datasets:**
    - Microsoft Malware Classification Challenge (Kaggle)
    - Malimg Dataset (GitHub)

7. Ransomware Detection

- **Description:** Detect ransomware behavior before encryption occurs
- **Datasets:**
  - Ransomware Detection Dataset (Kaggle)
  - VirusTotal API

8. Android Malware Detection

6

- **Description:** Identify malicious Android applications using permission and API call analysis
- **Datasets:**
  - Drebin Dataset
  - AndroZoo Dataset
  - Android Malware Dataset (Kaggle)

9. IoT Malware Detection

- **Description:** Detect malware targeting Internet of Things devices
- **Datasets:**
  - IoT-23 Dataset (UNB)
  - MedBIoT Dataset (GitHub)

10. PDF Malware Detection

- **Description:** Identify malicious PDF documents using structural analysis
- **Datasets:**
  - Contagio PDF Dataset
  - PDF Malware Dataset (Kaggle)

Section C: Phishing & Social Engineering

11. Phishing URL Detection

- **Description:** Classify phishing vs legitimate URLs using lexical and host-based features
- **Datasets:**
  - Phishing Dataset for ML (Kaggle)
  - PhishTank
  - Phishing Dataset (Hugging Face)

12. Phishing Email Detection

- **Description:** Detect phishing emails using NLP and content analysis
- **Datasets:**
  - Phishing Email Dataset (Kaggle)
  - Enron Email Dataset (CMU)
  - Nazario Phishing Corpus

13. Phishing Website Detection

- **Description:** Identify phishing websites by analyzing HTML structure and content
- **Datasets:**
  - UCI Phishing Websites Dataset

- Mendeley Data

14. Social Engineering Attack Detection

- **Description:** Detect social engineering attempts in communications
- **Datasets:**
  - SEAD Dataset (GitHub)
  - Social Media Spam Dataset (Kaggle)

15. SMS/Smishing Detection

- **Description:** Detect phishing via SMS messages (smishing)
- **Datasets:**
  - SMS Spam Collection (Kaggle)
  - Smishing Dataset (Kaggle)

Section D: Fraud Detection & Financial Security

16. Credit Card Fraud Detection

- **Description:** Detect fraudulent credit card transactions in real-time
- **Datasets:**
  - Credit Card Fraud Detection (Kaggle)
  - IEEE-CIS Fraud Detection Competition (Kaggle)

17. Insurance Fraud Detection

- **Description:** Identify fraudulent insurance claims using behavioral patterns
- **Datasets:**

- Vehicle Insurance Fraud (Kaggle)
- Insurance Claim Analysis (Kaggle)

18. Identity Theft Detection

- **Description:** Detect identity theft and account takeover attempts
- **Datasets:**
  - Identity Fraud Detection (Kaggle)
  - Account Takeover Samples (GitHub)

19. Money Laundering Detection

- **Description:** Identify suspicious financial transactions indicating money laundering
- **Datasets:**
  - Credit Card Transactions for AML (Kaggle)
  - IBM Financial Transactions Data

20. E-commerce Fraud Detection

- **Description:** Detect fraudulent e-commerce transactions and fake reviews
- **Datasets:**
  - E-commerce Fraud Detection (Kaggle)
  - Fake Reviews Dataset (Kaggle)

Section E: Web Application Security

21. SQL Injection Detection

- **Description:** Detect SQL injection attacks in web application inputs

- **Datasets:**
    - SQL Injection Detection Dataset (IEEE DataPort)
    - Cybersecurity Attack Dataset (Kaggle)

22. Cross-Site Scripting (XSS) Detection

- **Description:** Identify XSS vulnerabilities and attacks in web applications
- **Datasets:**
    - XSS Attack Dataset (GitHub)
    - OWASP WebGoat

23. Web Shell Detection

- **Description:** Detect malicious web shells on servers
- **Datasets:**
    - Web Shell Detection Dataset (Kaggle)
    - PHP Web Shells Collection (GitHub)

24. API Security & Abuse Detection

- **Description:** Detect API abuse, rate limiting violations, and malicious requests
- **Datasets:**
    - API Security Dataset (Kaggle)
    - VirusTotal API

Section F: Insider Threat & User Behavior

25. Insider Threat Detection

- **Description:** Detect malicious insider activities using user behavior analytics
- **Datasets:**
    - CERT Insider Threat Dataset (CMU)
    - Insider Threat Detection (Kaggle)

26. User Authentication & Behavior Biometrics

- **Description:** Verify users based on typing patterns, mouse movements, and other behavioral data
- **Datasets:**
    - Behavioral Biometrics Dataset (Kaggle)
    - Keystroke Dynamics Dataset (Kaggle)

27. Password Strength & Cracking Prediction

- **Description:** Predict password vulnerability and cracking difficulty
- **Datasets:**
    - RockYou Password List
    - Password Strength Classifier Dataset (Kaggle)

Section G: Advanced AI Security Topics

28. Adversarial Machine Learning Defense

- **Description:** Detect and defend against adversarial attacks on ML models
- **Datasets:**
    - Imagenet Adversarial Challenges (GitHub)
    - Cleverhans Adversarial Examples (GitHub)

29. Deepfake Detection

- **Description:** Identify AI-generated synthetic media including videos, audio, and images
- **Datasets:**
  - FaceForensics++ (GitHub)
  - Deepfake and Real Images (Kaggle)
  - CelebDF Dataset (GitHub)

30. Privacy-Preserving Federated Learning for Security

- **Description:** Build collaborative threat detection systems while preserving privacy
- **Datasets:**
  - LEAF Framework Benchmarks (GitHub)
  - Federated Learning Datasets (GitHub)

Additional Resources - Comprehensive Dataset Repositories

- **Kaggle Cybersecurity Datasets** - Curated collection of security-related datasets
- **UCI Machine Learning Repository** - Academic datasets for research
- **AWS Open Data Registry** - Cloud-hosted security datasets
- **NIST National Vulnerability Database** - Comprehensive vulnerability data
- **MITRE ATT&CK Framework** - Adversary tactics and techniques database

PART 2: Required Outputs from Each Team

---

1. Project Documentation Report (Required)

A. Executive Summary (1-2 pages)

- Problem statement and motivation
- Proposed solution overview
- Key results and achievements
- Real-world applications and impact

B. Technical Report (15-25 pages)

- Introduction and Literature Review
- Methodology and Architecture
- Data Preprocessing Pipeline
- Feature Engineering Process
- Model Selection Justification
- Experimental Setup
- Results and Analysis
- Discussion and Limitations
- Conclusion and Future Work

C. Code Documentation

- README file with setup instructions
- Requirements/dependencies list
- Inline code comments (minimum 20% of code)
- API documentation for functions
- User manual for running the system

2. Source Code Repository (Required)

A. Recommended Project Structure

```
project-root/
├── data/
│   ├── raw/
│   ├── processed/
│   └── README.md (data dictionary)
├── src/
│   ├── preprocessing/
│   ├── feature_engineering/
│   ├── models/
│   ├── training/
│   ├── evaluation/
│   └── utils/
├── notebooks/
│   ├── EDA.ipynb
│   ├── model_training.ipynb
│   └── results_analysis.ipynb
├── tests/
├── config/
├── requirements.txt
├── README.md
└── LICENSE
```

B. Minimum Code Components

- Data loading and preprocessing module
- Feature extraction/engineering module
- Model implementation/training script
- Evaluation and metrics calculation
- Prediction/inference pipeline
- Unit tests (minimum coverage: 60%)

3. Dataset & Data Handling (Required)

A. Data Documentation

- Source attribution and licensing information
- Data collection methodology description
- Data dictionary with feature descriptions
- Class distribution and imbalance analysis
- Data quality assessment report

B. Preprocessing Artifacts

- Cleaned dataset (CSV/Parquet format)
- Train/test/validation split files
- Feature scaling/normalization parameters
- Encoders and transformers (saved as pickle/joblib files)

4. Model Artifacts (Required)

A. Saved Models

- Trained model file (joblib/pickle/ONNX/H5 format)
- Model architecture configuration
- Hyperparameters used for training
- Training history logs (loss curves, metrics over epochs)

B. Model Evaluation Results

- Confusion matrix (visualized)
- Classification report (precision, recall, F1-score per class)
- ROC-AUC curves (for binary/multi-class)
- Cross-validation results

- Comparison table with baseline models

5. Visualizations & Results (Required - Minimum 8 Figures)

A. Exploratory Data Analysis (EDA) Plots

- Feature distributions (histograms/boxplots)
- Correlation heatmap
- Class imbalance visualization
- Missing values analysis

B. Model Performance Visualizations

- Training/validation loss curves
- Accuracy/precision/recall curves
- ROC curves and AUC scores
- Confusion matrix heatmaps
- Feature importance plots (for tree-based models)
- t-SNE/PCA visualizations (for clustering)

C. Comparative Analysis Charts

- Bar chart comparing different algorithms
- Performance vs complexity trade-off plot
- Inference time comparison table

6. Demonstration & Deployment (Required)

A. Working Demo Options (Choose One)

- Jupyter notebook with end-to-end pipeline
- Interactive demo (Streamlit/Gradio/Flask web app)

- Command-line interface for predictions
- Video demonstration (5-10 minutes)

B. Sample Predictions

- At least 10 test cases with predictions
- Explanation of predictions (SHAP/LIME values if applicable)
- False positive/negative case analysis

7. Presentation Materials (Required)

A. Slide Deck (15-20 slides)

- Title slide with team members
- Problem introduction
- Related work/literature
- Methodology overview
- System architecture diagram
- Implementation details
- Experimental results
- Live demo screenshots/video
- Challenges faced and solutions
- Future improvements
- Q&A preparation

B. Poster (Optional but Recommended)

- A0/A1 size research poster
- Visual summary of entire project
- Suitable for conference presentation

8. Additional Deliverables (Bonus Items)

- Docker container for reproducible environment
- API documentation (Swagger/OpenAPI)
- CI/CD pipeline configuration
- Model deployment on cloud platform
- Research paper draft (IEEE format)
- Video tutorial explaining the project

PART 3: Evaluation Rubric (Total: 20 Points)

Evaluation Overview: Each category is scored independently based on specific criteria. The total maximum score is 20 points. Detailed rubrics for each category follow below.

Category 1: Problem Definition & Motivation (1.5 points)

Score	Criteria
1.5 - Excellent	Clear, well-defined problem with strong real-world motivation; comprehensive background research; identifies gap in existing solutions; articulates impact and significance
1.2 - Good	Well-defined problem with adequate motivation; some background research; shows understanding of domain; minor gaps in problem framing

Score	Criteria
0.9 - Satisfactory	Problem stated but lacks clarity or depth; limited motivation; basic understanding shown; some ambiguity in scope
0.6 - Needs Improvement	Vague problem statement; weak motivation; minimal research; unclear objectives
0.3 - Poor	Problem poorly defined or missing; no motivation provided; no context given

Checklist:

- Problem statement is specific and measurable
- Real-world cybersecurity application identified
- Current challenges/gaps explained
- Target users/stakeholders defined
- Success criteria clearly stated

Category 2: Literature Review & Related Work (1.5 points)

Score	Criteria
1.5 - Excellent	Comprehensive review of 10+ relevant papers; critical analysis of existing methods; clear positioning of proposed approach; proper citations in IEEE format; identifies limitations of prior work

Score	Criteria
1.2 - Good	Review of 6-9 relevant papers; adequate analysis; good positioning; mostly proper citations; some critical insights
0.9 - Satisfactory	Review of 3-5 papers; basic summary; limited analysis; acceptable citations; minimal critical thinking
0.6 - Needs Improvement	Fewer than 3 papers reviewed; superficial summary; poor citations; no analysis
0.3 - Poor	No literature review or extremely minimal; no citations; irrelevant sources

Checklist:

Minimum 5 academic papers cited (10 for full marks)

Papers from reputable venues (IEEE, ACM, Springer, etc.)

Published within last 5 years (with exceptions for foundational work)

Critical comparison table of methods included

Clear explanation of how project advances state-of-the-art

Category 3: Dataset Selection & Preprocessing (2.5 points)

Score	Criteria
2.5 - Excellent	Appropriate dataset selected with justification; thorough EDA with multiple visualizations; comprehensive preprocessing pipeline; handles missing values, outliers, imbalance; data quality documented; train/test split properly done
2.0 - Good	Appropriate dataset; good EDA; solid preprocessing; most issues addressed; decent documentation
1.5 - Satisfactory	Acceptable dataset; basic EDA; standard preprocessing; some issues overlooked; minimal documentation
1.0 - Needs Improvement	Questionable dataset choice; minimal EDA; incomplete preprocessing; major issues ignored
0.5 - Poor	Inappropriate dataset; no EDA; little/no preprocessing; data quality problems

Checklist:

Dataset source properly attributed and licensed

Dataset size and characteristics documented

Exploratory analysis includes: distributions, correlations, missing values, outliers

Class imbalance identified and addressed (SMOTE, undersampling, etc.)

Feature scaling/normalization applied appropriately

Train/validation/test split ratio justified (e.g., 70/15/15)

Data leakage prevention measures taken

All preprocessing steps reproducible

Category 4: Feature Engineering (2.5 points)

Score	Criteria
2.5 - Excellent	Creative and domain-specific features engineered; strong justification for each feature; feature selection using statistical/ML methods; dimensionality reduction where appropriate; feature importance analyzed; ablation study on features
2.0 - Good	Well-engineered features; good justifications; proper feature selection; importance analyzed
1.5 - Satisfactory	Basic feature engineering; some justifications; minimal selection; limited analysis
1.0 - Needs Improvement	Minimal feature engineering; weak justifications; no selection process
0.5 - Poor	No feature engineering; raw features used without thought

Checklist:

Domain knowledge incorporated into features

New features created (not just raw dataset columns)

Categorical features properly encoded (one-hot, label, target encoding)

Temporal/spatial features extracted if applicable

Feature selection method applied (mutual info, chi-square, RFE, etc.)

Number of final features documented and justified

Feature correlation with target analyzed

Ablation study showing impact of feature sets on performance

Category 5: Model Selection & Implementation (3 points)

Score	Criteria
3.0 - Excellent	Multiple models compared (minimum 5); deep learning model included where appropriate; model architectures well-designed; hyperparameter tuning systematic (grid/random/bayesian); regularization techniques applied; overfitting prevention strategies implemented
2.4 - Good	3-4 models compared; decent architectures; some tuning; basic regularization
1.8 - Satisfactory	2-3 models compared; standard implementations; minimal tuning
1.2 - Needs Improvement	Only 1-2 models; basic implementation; no tuning
0.6 - Poor	Single model; poor implementation; no optimization

Checklist:

Minimum 3 different ML algorithms implemented (5 for excellent)

At least one deep learning model (if applicable to problem)

Baseline model included (dummy classifier, simple threshold)

Hyperparameter search space defined and executed

Cross-validation strategy appropriate (stratified k-fold, time-series split)

Regularization used (L1/L2, dropout, early stopping)

Ensemble methods considered (voting, stacking, boosting)

Model complexity vs performance trade-off analyzed

Score	Criteria
3.0 - Excellent	Comprehensive metrics appropriate for problem type (accuracy, precision, recall, F1, ROC-AUC, PR-AUC); per-class performance reported; statistical significance testing; confidence intervals; comparison with state-of-the-art; error analysis with examples
2.4 - Good	Good set of metrics; solid reporting; some comparisons; basic error analysis
1.8 - Satisfactory	Standard metrics reported; basic results; minimal comparison
1.2 - Needs Improvement	Limited metrics; incomplete reporting; no

Score	Criteria
	comparisons
0.6 - Poor	Only accuracy reported; incomplete results; misleading metrics

Checklist:

- Metrics appropriate for dataset (imbalanced? multi-class? multi-label?)
- Confusion matrix included and interpreted
- Precision, Recall, F1-score reported per class
- ROC-AUC curve (binary) or macro/micro averages (multi-class)
- Cross-validation results with mean and std deviation
- Statistical significance tested (t-test, Wilcoxon, etc.)
- Comparison table with baseline and related work
- False positive/negative cases analyzed qualitatively
- Training time and inference time benchmarked
- Memory footprint and computational cost documented

Category 7: Code Quality & Reproducibility (2 points)

Score	Criteria
2.0 - Excellent	Clean, modular, PEP8 compliant code; comprehensive documentation; version control (Git) with meaningful commits; requirements.txt and environment.yml; unit

Score	Criteria
	tests with >70% coverage; follows software engineering best practices; code reusable and extensible
1.6 - Good	Well-organized code; good documentation; version control; some tests; mostly readable
1.2 - Satisfactory	Functional code; basic comments; version control present; minimal tests
0.8 - Needs Improvement	Messy code; sparse comments; poor organization; no tests
0.4 - Poor	Unreadable code; no documentation; doesn't run; missing files

Checklist:

Code runs without errors from clean environment

Functions are small, focused, and single-purpose

Meaningful variable and function names used

Magic numbers replaced with named constants

Configuration separated from code (config files, argparse)

Error handling and logging implemented

Git repository with minimum 10 commits

README with installation and usage instructions

requirements.txt or environment.yml included

Unit tests for critical functions

No hardcoded paths or sensitive information

Category 8: Technical Report & Documentation (2 points)

Score	Criteria
2.0 - Excellent	Professional, well-structured report (20+ pages); clear writing with proper grammar; all sections complete; figures and tables properly labeled; references in correct format; appendix with supplementary material; executive summary compelling
1.6 - Good	Well-written report (15-19 pages); most sections complete; good formatting; proper references
1.2 - Satisfactory	Adequate report (10-14 pages); all sections present; acceptable writing; some formatting issues
0.8 - Needs Improvement	Short report (<10 pages); missing sections; writing issues; poor formatting
0.4 - Poor	Incomplete or missing report; major deficiencies; unreadable

Checklist:

Report follows provided template/structure

Abstract/executive summary included (250-300 words)

Introduction provides sufficient background

Methodology section detailed enough for reproduction

All figures have captions and referenced in text

Tables formatted professionally

Mathematical notation properly typeset

References in consistent citation style (IEEE recommended)

Page numbers, headers, footers included

Spell-check and grammar-check performed

Appendix includes additional results, code snippets

Category 9: Presentation & Communication (1.5 points)

Score	Criteria
1.5 - Excellent	Engaging, professional presentation (15-20 min); clear visual aids; confident delivery; all team members participate; excellent Q&A handling; demo works flawlessly; time management perfect
1.2 - Good	Solid presentation; good visuals; mostly confident; most participate; good demo; minor timing issues
0.9 - Satisfactory	Adequate presentation; acceptable visuals; some participation; demo works; some timing issues
0.6 - Needs Improvement	Weak presentation; poor visuals; dominated by one person; demo issues; overtime/undertime

Score	Criteria
0.3 - Poor	Disorganized; unprepared; no demo; major problems

Checklist:

- Slide design professional and consistent
- Text on slides minimal (bullet points, not paragraphs)
- High-quality images and diagrams
- Demo video or live demo prepared
- Each member presents approximately equal time
- Practice session completed before presentation
- Backup plan for technical difficulties
- Anticipated questions prepared with answers
- Presentation within time limit (+/- 1 minute)
- Clear take-home message delivered

Category 10: Innovation & Originality (1 point)

Score	Criteria
1.0 - Excellent	Novel approach or significant improvement over existing methods; creative feature engineering; innovative model architecture; unique application to cybersecurity domain; potential for publication or patent

Score	Criteria
0.8 - Good	Some original ideas; interesting combination of methods; creative application; shows independent thinking
0.6 - Satisfactory	Minor innovations; standard approach with small twists; some creativity evident
0.4 - Needs Improvement	Mostly follows tutorials/examples; little originality; cookie-cutter approach
0.2 - Poor	Complete replication of existing work; no original contribution

Checklist:

Novel contribution clearly identified

Approach differs from existing solutions in literature

Creative problem-solving demonstrated

Unique insights or observations shared

Potential for future research or real-world deployment discussed

PART 4: Scoring Summary & Grade Boundaries

Scoring Summary Sheet

Grade Boundaries

PART 5: Submission Checklist & Timeline

Final Submission Package Should Include

1. Technical Report (PDF)

File name format: TeamName_TechnicalReport.pdf

Page limit: 25 pages maximum (excluding appendix)

2. Source Code Repository (GitHub/GitLab link)

Public or private with instructor access

Contains all source code, notebooks, and configurations

Includes README with setup instructions

3. Presentation Slides (PDF or PowerPoint)

File name format: TeamName_Presentation.pptx/pdf

15-20 slides maximum

4. Demo Video or Live Demo Link

YouTube (unlisted) or recorded video file

5-10 minutes maximum duration

Shows working system with sample predictions

5. Trained Model Files

Uploaded to cloud storage or included in repo

Include model architecture and training config

6. Dataset Documentation

Data dictionary

Preprocessing scripts

Train/test split files (if dataset not public)

7. Team Contribution Statement

Each member's contributions listed

Signed by all team members

Percentage of effort distribution

8. Peer Evaluation Form (confidential)

Rate each team member's participation

Submit individually to instructor

Key Tips for Success

- Start with preprocessed datasets (like those on Kaggle) if you are a beginner
- Use feature engineering to extract meaningful patterns from raw security data
- Consider ensemble methods combining multiple machine learning models for better performance
- Always evaluate using precision, recall, and F1-score rather than just accuracy due to class imbalance issues
- Implement real-time detection pipelines for production readiness

- Document your methodology thoroughly for reproducibility
- Practice your presentation multiple times before the final delivery
- Version control your code frequently with meaningful commit messages

---